School: ..... Campus: .....

Academic Year: ..... Subject Name: ..... Subject Code: .....

Semester: ..... Program: ..... Branch: ..... Specialization: .....

Date: .....

Applied and Action Learning

(Learning by Doing and Discovery)

Name of the Experiement : Proxy in Practice – Using Libraries and Upgradable Contracts

Objective/Aim:

To study the use of libraries and proxy patterns for creating upgradeable smart contracts in Solidity.

Apparatus/Software Used:

- Computer/Laptop
- Internet connection
- Remix IDE (Solidity compiler)
- Web browser

Theory/Concept:

Libraries in Solidity:

Libraries are reusable pieces of code that can be used by multiple smart contracts. They help in reducing code duplication and improving efficiency. Libraries are deployed once and their functions can be called by other contracts, making the code modular and gas-efficient.

Proxy and Upgradeable Contracts:

Upgradeable contracts use a proxy pattern where one contract (proxy) stores data and delegates logic execution to another contract (implementation). This allows the logic to be updated without changing the contract address or losing stored data. It is useful for fixing bugs and improving features after deployment.

Procedure:

1. Open Remix IDE in a web browser.
2. Create a Solidity file.
3. Write a library with reusable functions.
4. Create a main contract that uses the library.
5. Implement proxy contract structure for upgradeability.
6. Compile the contracts.
7. Deploy contracts using Remix environment.
8. Test functions and observe behavior.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// Proxy Contract
contract Proxy {
    address public implementation;

    constructor(address _impl) {
        implementation = _impl;
    }

    // Upgrade to new implementation
    function upgrade(address _newImpl) public {
        implementation = _newImpl;
    }

    // Receive Ether directly
    receive() external payable {}

    // Delegate call to implementation
    fallback() external payable {
        (bool success, ) = implementation.delegatecall(msg.data);
        require(success, "Call failed");
    }
}
```

Observation Table:

Concept Used	Description	Result
Library	Reusable code module	Reduced duplication
Proxy Pattern	Separates logic and data	Upgrade possible
Delegatecall	Calls implementation contract	Logic executed correctly
Deployment	Contracts deployed in Remix	Successful execution

ASSESMENT

Rubrics	Full Mark	Marks Obtained	Remarks
Concept	10		
Planning and Execution/ Practical Simulation/ Programming	10		
Result and Interpretation	10		
Record of Applied and Action Learning	10		
Viva	10		
Total	**50**		